

HACKATHON · TRACK 02 · GENAI DATA HARMONIZATION & QUALITY GUARDRAILS

GoldenRecord

Fabric tells you your data is bad. GoldenRecord fixes it:
safely, by root cause, with an audit trail.

Team GoldenRecord · [Event date]

Vinod Atmakur · Anand Topu · Yogesh Godwade · Koushik Das · Ravi Chander Kanikala · Dilip Divakaran

THE PROBLEM

Bad data reaches the report before anyone notices



Duplicates

“IBM Corp” and “International Business Machines” counted as two vendors across ERPs.



Invalid values

A date of birth after the account open date. Amounts far out of range.



Inconsistent formats

“Bangalore” vs “Bengaluru”.
Mixed date, currency and phone formats.

The result: leaders stop trusting the numbers, and every team rebuilds its own.

Track brief: “50,000+ records, half of them garbage.”

OUR POINT OF VIEW

Fabric tells you your data is bad. GoldenRecord fixes it.

Quarantined with a reason, repaired by rules learned once, resolved into one golden vendor, and approved by root cause.

BUILT ON FABRIC, NOT BESIDE IT

Fabric detects. We fix and resolve.

Job	Native Fabric (we use it)	GoldenRecord adds
Detect	Materialized lake view constraints	Quarantine view: every dropped row with its rules
Alert	Activator on a per-source score view	Scores tied to root causes a steward can fix
Fix	—	Fixes authored once by AI Functions, versioned
Resolve	— (fuzzy merge is a join, not MDM)	Golden records: survivorship, explain, unmerge
Steward	Power BI translytical task flows	Pattern queue: one click, thousands of records
Protect	OneLake security, Key Vault	PII tokenized from Bronze; raw only in a vault

Purview is optional: the same rule IDs feed its scorecard where the tenant has it. Nothing in the solution runs outside Fabric.

THE GUARDRAIL LOOP

Detect, explain, fix once, approve by pattern

01

Detect

Native MLV constraints
check every row against one
rule catalog.

02

Explain

Dropped rows are quarantined
with the rules they failed and a
root cause.

03

Fix once

AI Functions author one fix
per pattern; it is validated,
versioned and reused.

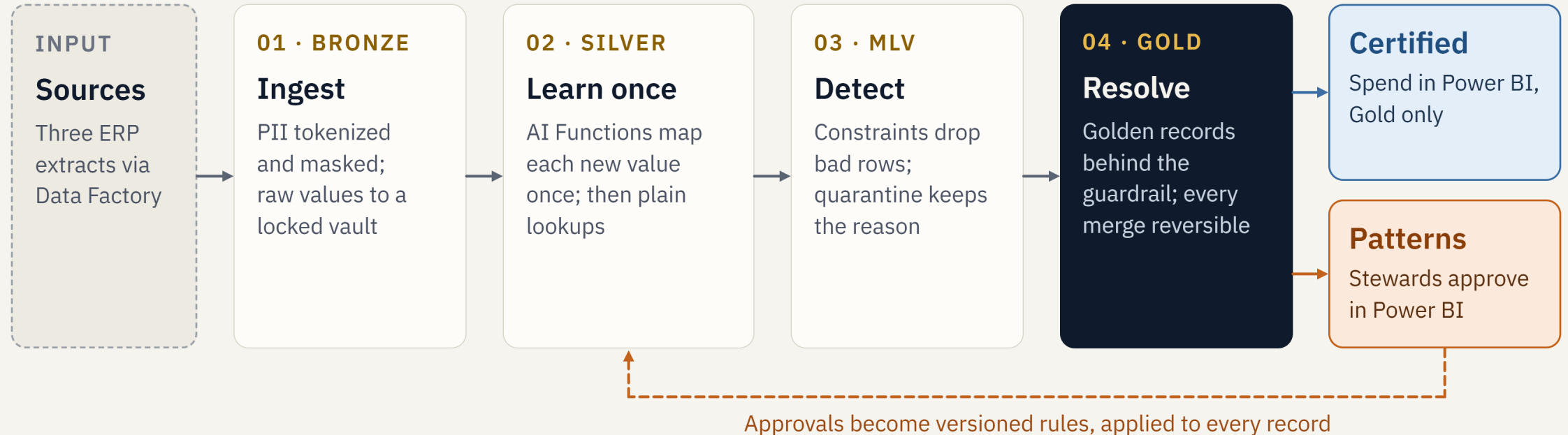
04

Approve

Stewards approve by root
cause in Power BI: one click,
thousands of records.

Supporting proof: golden vendor records with survivorship put 99.1% of Gold spend on the right vendor.

How GoldenRecord works



05 Native governance OneLake security · Key Vault · Activator per-source alerts · MLV lineage · Purview optional

THE GUARDRAIL

AI proposes once, rules decide every time

- 1 Only distinct values never seen before reach AI Functions.
- 2 A proposal is accepted only if its target already exists.
- 3 Confident and valid: into the versioned library. Otherwise: a steward pattern.
- 4 Pairs with two valid tax IDs never merge, whatever the AI says.
- 5 Every mapping, fix and merge records who decided it, and can be undone.

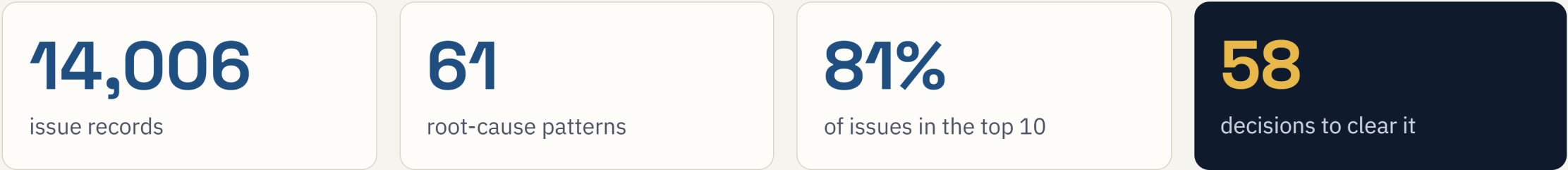
0

AI calls on a rerun, with identical output

107,525

calls a run if the LLM looked at every row and pair

Decide by root cause, not by row



Pattern (example)	One click does
ERP_B: 320 invoices with blank currency	Use the vendor's country currency
144 pairs: same name, phone, email domain	Merge all, now and on future loads
106 records with city "Calcutta"	Map to Kolkata for good
2,266 invoices: vendor not in master	Keep quarantined; notify each owner

PII is protected from Bronze, not at the gate



Tokenized at ingestion

Phone, email and bank account become keyed tokens and masked values. The key lives in Key Vault.



OneLake security

Raw values sit only in a vault table that one custodian role can read.



Matching without seeing

Dedup compares tokens; the LLM and stewards only ever see masked values.



Certified, Gold only

The semantic model reads only Gold. Purview labels and lineage are added where available.

MEASURED ON THE SAME SEEDED DATASET

Native constraints vs GoldenRecord

52,438 records · 3,000 true vendors	MLV only	Rules only	+58 decisions	LLM per row
Reason per bad row	none	all	all	–
Records repaired	0	0	1,185	–
Duplicates merged (1,743)	0	1,549	1,727	–
Match recall	0	0.915	0.992	not run
Spend on right vendor	split	90.5%	99.1%	–
LLM calls per run	0	0	0	107,525
Same output on rerun	yes	yes	yes	no

python -m goldenrecord ablation · precision 1.00 throughout · steward simulated · LLM-per-row accuracy to be measured on AI Functions

WHY US

Where similar teams will stop

"Purview + MLV" teams

They detect and score. They repair 0 records and merge 0 duplicates. We use the same native detection, then fix.

"LLM cleanser" teams

107,525 AI calls a run, different answers tomorrow, nothing to audit. We call it once per pattern: 0 on a rerun.

"Fuzzy dedup" teams

Duplicates found, but no survivorship, no stable keys, no explain or unmerge. Ours are reversible.

[What we have seen in real data-quality work, in one line, from the team]

TRY TO BREAK IT

Three failure modes, three tested answers

A wrong rule

Impact preview by source. Over 5% of the table: blocked. Accepted rules start in shadow mode, quarantine nothing, and roll back by retiring a version.

A false merge

Flagged automatically: two tax IDs, countries or bank accounts. Explain shows who merged it. Unmerge adds a versioned cannot-link that holds forever.

25,000 issues

Grouped by root cause: 14,006 issues became 61 patterns, cleared with 58 decisions. Each decision is a versioned rule for future loads too.

TECHNOLOGY

Every component is a Fabric item

Job	Fabric service	Used for
Ingest	Data Factory pipeline	Land and orchestrate
Store, protect	OneLake, Lakehouse, OneLake security	Medallion layers, PII vault
Learn, resolve	Notebooks, AI Functions	Mappings once; grey-zone pairs
Detect	Materialized lake views	Constraints, quarantine view
Anomalies, learning	Fabric Data Science: SynapseML, MLflow	Isolation Forest, matcher model
Steward	Power BI, User Data Functions, SQL database	Translytical task flows
Alert, report	Activator, Power BI (Direct Lake)	Source alerts, certified spend

Purview optional. Outside Fabric there is only an offline dev harness (Docker) running the same tested package, for development and a backup demo.

AGAINST THE BRIEF

Every required deliverable, covered

#	Brief deliverable	How GoldenRecord delivers it
1	Fabric harmonization service	Bronze → Silver (MLVs) → Gold, survivorship, stable keys
2	LLM classification and matching	AI Functions: decision, band, reason; once per pattern
3	Human-in-the-loop review queue	Pattern queue, translytical write-back, versioned library
4	5+ DQ rules, anomaly alerts	9 MLV constraints + Isolation Forest; Activator per source
5	Sensitivity protection	Tokenized from Bronze, OneLake security; Purview labels if available
6	Certified model, spend report	Direct Lake on gold_spend_certified only

THE DEMO

Five minutes, five moments

0:00

Fabric finds it

MLV lineage: 12,164
rows dropped. No
reasons, no fixes.

1:00

We explain it

Same rows,
quarantined with
reasons: 14,006 issues,
61 patterns.

2:00

One click

Approve a pattern in
Power BI; 320 invoices
fixed on the rerun.

3:00

Break it

A judge types a bad
rule: blocked. Unmerge
a false merge.

4:00

Money

Certified spend, 99.1%
on the right vendor;
Activator alert.

THE TEAM

Team GoldenRecord

Vinod Atmakur

Product owner & quality architect

20+ years in delivery and quality architecture: payments, fintech, healthcare

Anand Topu

AI Engineer

[One line of relevant experience]

Yogesh Godwade

AI Engineer

[One line of relevant experience]

Koushik Das

AI Engineer

[One line of relevant experience]

Ravi Chander Kanikala

AI Engineer

[One line of relevant experience]

Dilip Divakaran

AI Engineer

[One line of relevant experience]

BUILD PLAN

Two weeks, four sprints

DAYS 1–3

Foundation

- Tested core and agents (done)
- Workspace, Lakehouse, Key Vault
- Bronze with tokenized PII

DAYS 4–7

Detect & learn

- AI Functions learn mappings
- MLV constraints, quarantine
- Measure AI accuracy

DAYS 8–11

Resolve & steward

- Golden records in Fabric
- Translytical pattern queue
- Activator per-source alerts

DAYS 12–14

Prove & demo

- Certified spend report
- Ablation rerun on Fabric
- Rehearse break-it scenarios

GOLDENRECORD

Fabric finds it. GoldenRecord
fixes it.

Thank you. Questions?